MyCodeSchool

Exerciții — recursivitate

Recapitulare · C++ · 12 algoritmi, 60 probleme · David

`algoritmi.h` e doar de studiu. Are înăuntru algoritmii de mai jos, gata scriși, cu urme de trasare a stivei de apeluri. Îl citești, îl închizi, apoi **scrii tu** algoritmul în headerul tău. Dacă te blochezi, îl trasezi pe hârtie, nu-l copiezi.

Regula fișei:

- **fiecare funcție se rezolvă RECURSIV:** fără `for`, `while` sau `do` în funcțiile cerute și în funcțiile ajutătoare pe care le scrii;
- poți scrie o funcție ajutătoare cu parametri în plus (un acumulator, doi indici); funcția cerută o apelează, cu semnătura dată în fișă;
- un vector se primește ca `int v[], int n`; `cout` apare doar în funcțiile care cer să afișeze, în rest funcțiile **întorc** rezultatul.

Cum lucrezi la fișă:

- **pentru fiecare algoritm îți faci headerul lui** — numele e scris în titlul fiecărei secțiuni;
- sus în header scrii **tu** algoritmul de bază, apoi cele 5 funcții;
- garda se scrie fără cratimă: `FACTORIAL_PUTERE_H_INCLUDED`, nu `FACTORIAL-PUTERE_H_INCLUDED`;
- la problemele **«Ce afișează / ce întoarce»** răspunzi mai întâi pe hârtie, apel cu apel; abia apoi copiezi funcția în header și rulezi, ca să te verifici;
- în `main.cpp` incluzi headerul la care lucrezi, apoi chemi funcțiile pe exemplele din fișă;
- build cu **Ctrl+Shift+B** (pe Mac **Cmd+Shift+B**; task-ul `build` din `.vscode/tasks.json`), rulezi din terminal cu `./main` (Windows `cmd`: `main`; Code::Blocks: **F9**).

```
Code::Blocks (Windows): File → New → Project → Console application (C++)
  main.cpp al proiectului = cel din repo
  headerele: în același folder, adăugate cu Project → Add files
  F9 = Build and run
```

Reguli:

- **Nu ai voie** să incluzi `algoritmi.h`, să chemi ceva din el sau să copiezi din el.
- **Ai voie** să chemi funcțiile **tale**, din headerele tale, oricâte, în orice combinație.
- **Nu ai voie:** `string`, `vector` din bibliotecă, `#include <algorithm>`.

1. Factorial și putere · `factorial-putere.h`

Studiezi din `algoritmi.h`: `factorial, putere`

Suma până la n

1.1

Scrie o funcție care întoarce suma 1 + 2 + … + n.

`int sumaPanaLa(int n)`

```
5    ->  15
1    ->  1
0    ->  0
100  ->  5050
```

Produsul numerelor impare

1.2

Scrie o funcție care întoarce produsul numerelor impare de la 1 la n. Pentru n = 0 produsul este 1.

`int produsImpare(int n)`

```
7  ->  105
8  ->  105
1  ->  1
0  ->  1
```

Puterea modulo

1.3

Scrie o funcție care întoarce baza la puterea exp, modulo m (baza * rest la fiecare pas, ca să nu depășești int; m ≤ 40000).

`int putereMod(int baza, int exp, int m)`

```
baza = 2 , exp = 10 , m = 1000  ->  24
baza = 7 , exp = 0 , m = 5      ->  1
baza = 5 , exp = 3 , m = 13     ->  8
baza = 0 , exp = 0 , m = 7      ->  1
baza = 3 , exp = 4 , m = 1      ->  0
```

Puterea rapidă

1.4

Scrie o funcție care calculează baza la exp coborând cu exp / 2 în loc de exp - 1: dacă exp e par, rezultatul e pătratul puterii cu exp / 2.

`int putereRapida(int baza, int exp)`

```
baza = 2 , exp = 10  ->  1024
baza = 3 , exp = 0   ->  1
baza = 5 , exp = 1   ->  5
baza = 7 , exp = 5   ->  16807
baza = 2 , exp = 30  ->  1073741824
```

Prima putere a lui 2

1.5

Scrie o funcție care întoarce cea mai mică putere a lui 2 mai mare sau egală cu n. Pentru n = 0 și n = 1 întoarce 1.

`int putereDe2(int n)`

```
5   ->  8
16  ->  16
17  ->  32
1   ->  1
0   ->  1
```

2. Suma și numărul cifrelor · `cifre-recursiv.h`

Studiezi din `algoritmi.h`: `sumaCifre, nrCifre`

Cifra maximă 2.1

Scrie o funcție care întoarce cea mai mare cifră a unui număr.

`int cifraMaxima(int n)`

```
4738  ->  8
7     ->  7
0     ->  0
1000  ->  1
```

Câte cifre pare 2.2

Scrie o funcție care întoarce câte cifre pare are un număr n > 0.

`int cifrePare(int n)`

```
4738  ->  2
1357  ->  0
2000  ->  4
7     ->  0
```

Cifra de control 2.3

Scrie o funcție care înlocuiește numărul cu suma cifrelor lui, până rămâne o singură cifră, și întoarce acea cifră.

`int cifraDeControl(int n)`

```
9875  ->  2
99    ->  9
10    ->  1
6     ->  6
0     ->  0
```

Cifra de pe poziția k 2.4

Scrie o funcție care întoarce cifra de pe poziția k, numărată de la dreapta începând de la 1, a unui număr n > 0. Dacă numărul nu are atâtea cifre, întoarce -1.

`int cifraDinPozitia(int n, int k)`

```
n = 48723 , k = 1  ->  3
n = 48723 , k = 3  ->  7
n = 48723 , k = 5  ->  4
n = 4738 , k = 5   ->  -1
n = 7 , k = 1      ->  7
```

Câte au suma s 2.5

Scrie o funcție care întoarce câte numere de la 1 la n au suma cifrelor egală cu s.

`int cuSumaCifrelor(int n, int s)`

```
n = 20 , s = 5    ->  2
n = 9 , s = 9     ->  1
n = 1 , s = 1     ->  1
n = 1000 , s = 1  ->  4
n = 0 , s = 5     ->  0
```

3. Oglinditul cu acumulator · `oglindit.h`

Studiezi din `algoritmi.h`: `oglindit, estePalindrom`

Ce întoarce f? 3.1

Pe hârtie: ce întoarce fiecare apel de mai jos? Urmărește stiva de apeluri. Apoi copiază funcția în header și verifică rulând.

```
int f(int n, int r)
{
    if (n == 0)
    {
        return r;
    }
    if (n % 2 == 0)
    {
        return f(n / 10, r * 10 + n % 10);
    }
    return f(n / 10, r);
}
```

```
f(4738, 0)  ->  ?
f(2468, 0)  ->  ?
f(1357, 0)  ->  ?
f(25, 7)    ->  ?
f(0, 5)     ->  ?
```

Suma cu oglinditul 3.2

Scrie o funcție care întoarce suma dintre un număr și oglinditul lui.

`int sumaCuOglinditul(int n)`

```
123  ->  444
120  ->  141
7    ->  14
0    ->  0
```

Lipește oglinditul 3.3

Scrie o funcție care lipește la dreapta numărului cifrele lui în ordine inversă, ca să obții un palindrom cu număr par de cifre (n ≤ 9999, ca rezultatul să încapă în int).

`int lipesteOglinditul(int n)`

```
123  ->  123321
120  ->  120021
7    ->  77
0    ->  0
```

Oglindit fără zerouri 3.4

Scrie o funcție care întoarce oglinditul numărului, sărind peste cifrele 0.

`int oglinditFaraZerouri(int n)`

```
1203  ->  321
1000  ->  1
4738  ->  8374
5050  ->  55
0     ->  0
```

Adună cu oglinditul 3.5

Scrie o funcție care adună repetat numărul cu oglinditul lui, până obține un palindrom, și întoarce în câți pași a ajuns. Dacă numărul e deja palindrom, întoarce 0. Se dau doar numere n ≤ 200, fără 89, 98, 177, 187 și 196 (la ele sumele nu mai încap în int).

`int pasiPanaLaPalindrom(int n)`

```
87   ->  4
56   ->  1
121  ->  0
195  ->  4
0    ->  0
```

4. Cel mai mare divizor comun · `cmmdc.h`

Studiezi din `algoritmi.h` : `cmmdc`

Cel mai mic multiplu comun 4.1

Scrie o funcție care întoarce cmmmc al lui a și b, folosind cmmdc.

`int cmmmc(int a, int b)`

```
a = 12 , b = 18  ->  36
a = 7 , b = 13   ->  91
a = 5 , b = 5    ->  5
a = 1 , b = 9    ->  9
```

Numitorul fracției ireductibile 4.2

Scrie o funcție care primește o fracție a / b și întoarce numitorul ei după simplificare.

`int numitorIreductibil(int a, int b)`

```
a = 12 , b = 18  ->  3
a = 7 , b = 7    ->  1
a = 4 , b = 9    ->  9
a = 0 , b = 5    ->  1
```

Prime între ele 4.3

Scrie o funcție care spune dacă două numere sunt prime între ele.

`bool suntPrimeIntreEle(int a, int b)`

```
a = 8 , b = 15   ->  true
a = 12 , b = 18  ->  false
a = 1 , b = 1    ->  true
a = 1 , b = 9    ->  true
a = 10 , b = 25  ->  false
```

Câți pași are Euclid 4.4

Scrie o funcție care întoarce de câte ori se face împărțirea cu rest până când restul devine 0.

`int pasiEuclid(int a, int b)`

```
a = 48 , b = 18  ->  3
a = 13 , b = 8   ->  5
a = 7 , b = 7    ->  1
a = 5 , b = 0    ->  0
```

Indicatorul lui Euler

4.5

Scrie o funcție care întoarce câte numere de la 1 la n sunt prime cu n.

`int indicatorEuler(int n)`

```
12  ->  4
7   ->  6
10  ->  4
2   ->  1
1   ->  1
```

5. Fibonacci · `fibonacci.h`

Studiezi din `algoritmi.h`: `fib, fibRapid`

Câte apeluri

5.1

Scrie o funcție care întoarce de câte ori este chemată fib în total (apelul inițial inclus) când calculezi fib(n) pe varianta lentă.

`int apeluriFib(int n)`

```
0   ->  1
1   ->  1
4   ->  9
10  ->  177
20  ->  21891
```

Combinări

5.2

Scrie o funcție care întoarce combinări de n luate câte k, cu C(n, k) = C(n - 1, k - 1) + C(n - 1, k).

`int combinari(int n, int k)`

```
n = 5 , k = 2    ->  10
n = 6 , k = 0    ->  1
n = 4 , k = 4    ->  1
n = 10 , k = 3   ->  120
n = 20 , k = 10  ->  184756
```

Șirul cu dublu

5.3

Scrie o funcție care întoarce termenul de pe poziția n (numărat de la 0) al șirului care începe cu a, b, iar fiecare termen următor e dublul celui dinaintea lui plus cel dinaintea acestuia: t(n) = 2 · t(n - 1) + t(n - 2). Se dau doar valori pentru care termenii încap în int.

`int termenul(int n, int a, int b)`

```
n = 5 , a = 2 , b = 3   ->  111
n = 0 , a = 4 , b = 7   ->  4
n = 1 , a = 4 , b = 7   ->  7
n = 6 , a = 1 , b = 1   ->  99
n = 10 , a = 0 , b = 1  ->  2378
```

Tribonacci

5.4

Scrie o funcție care întoarce al n-lea termen (de la 0) al șirului 0, 0, 1, în care fiecare termen următor e suma celor trei dinaintea lui.

`int tribonacci(int n)`

```
10  ->  81
0   ->  0
1   ->  0
2   ->  1
20  ->  35890
```

Urcă scările

5.5

Poți urca o treaptă sau două deodată. Scrie o funcție care întoarce în câte moduri urci n trepte. Pentru n = 0 există un singur mod: nu urci.

`int moduriScari(int n)`

```
0   ->  1
1   ->  1
2   ->  2
4   ->  5
10  ->  89
30  ->  1346269
```

6. Afișare înainte și după apel · `afisare-apel.h`

Studiezi din `algoritmi.h`: `numaratoareInversa, numaratoareDirecta, cifreInOrdine, cifreInvers`

Ce afișează f?

6.1

Pe hârtie: ce afișează fiecare apel de mai jos? Urmărește stiva de apeluri. Apoi copiază funcția în header și verifică rulând.

```
void f(int n)
{
    if (n > 0)
    {
        cout << n << " ";
        f(n - 1);
        cout << n << " ";
    }
}
```

```
f(3)  ->  ?
f(1)  ->  ?
f(4)  ->  ?
f(0)  ->  ?
```

Ce afișează g?

6.2

Pe hârtie: ce afișează fiecare apel de mai jos? Desenează arborele de apeluri. Apoi copiază funcția în header și verifică rulând.

```
void g(int n)
{
    if (n > 1)
    {
        g(n - 2);
        cout << n << " ";
        g(n - 2);
    }
}
```

```
g(5)  ->  ?
g(6)  ->  ?
g(2)  ->  ?
g(1)  ->  ?
```

Muntele

6.3

Scrie o funcție care afișează numerele de la k la n, apoi înapoi până la k, fără să repete n. Dacă k > n nu afișează nimic.

`void munte(int k, int n)`

```
k = 1 , n = 4  ->  1 2 3 4 3 2 1
k = 3 , n = 5  ->  3 4 5 4 3
k = 1 , n = 1  ->  1
k = 2 , n = 1  ->  (nimic)
```

Dus-întors

6.4

Scrie o funcție care afișează cifrele unui număr n > 0 de la dreapta la stânga, apoi de la stânga la dreapta.

`void dusIntors(int n)`

```
123   ->  3 2 1 1 2 3
4738  ->  8 3 7 4 4 7 3 8
10    ->  0 1 1 0
7     ->  7 7
```

Pare înainte, impare după

6.5

Scrie o funcție care afișează întâi cifrele pare ale unui număr n > 0, de la dreapta la stânga, apoi cifrele impare, de la stânga la dreapta.

`void pareApoiImpare(int n)`

```
4738   ->  8 4 7 3
12345  ->  4 2 1 3 5
2468   ->  8 6 4 2
1357   ->  1 3 5 7
7      ->  7
```

7. Suma și maximul unui vector · `suma-maxim-vector.h`

Studiezi din `algoritmi.h`: `sumaVector, maximVector`

Câte elemente pare

7.1

Scrie o funcție care întoarce câte elemente pare are vectorul.

`int cateElementePare(int v[], int n)`

```
4 7 2 9 6  ->  3
1 3 5      ->  0
8          ->  1
7          ->  0
```

Produsul elementelor

7.2

Scrie o funcție care întoarce produsul elementelor vectorului.

`int produsElemente(int v[], int n)`

```
2 3 4  ->  24
5      ->  5
1 1 1  ->  1
7 0 3  ->  0
```

Unde e maximul

7.3

Scrie o funcție care întoarce poziția maximului (de la 0). Dacă maximul apare de mai multe ori, o întoarce pe prima.

`int pozitiaMaximului(int v[], int n)`

```
3 9 2 9 1  ->  1
5          ->  0
1 2 3      ->  2
4 4 4      ->  0
```

E crescător?

7.4

Scrie o funcție care spune dacă vectorul e ordonat crescător (valorile egale sunt permise).

`bool esteCrescator(int v[], int n)`

```
1 3 7  ->  true
1 7 3  ->  false
2 2 5  ->  true
5      ->  true
```

Recordurile

7.5

Scrie o funcție care întoarce câte elemente sunt strict mai mari decât toate cele dinaintea lor. Primul element se numără.

`int cateRecorduri(int v[], int n)`

```
3 1 4 1 5 9 2 6  ->  4
4 3 2 1          ->  1
1 2 3            ->  3
2 2 2            ->  1
5                ->  1
```

8. Căutarea binară · `cautare-binara.h`

Studiezi din `algoritmi.h` : `cautareBinara`

Apare valoarea? 8.1

Primești un vector ordonat crescător și o valoare x. Scrie o funcție care spune dacă x apare în vector, prin căutare binară.

`bool apare(int v[], int n, int x)`

```
2 5 9 14 20 , x = 9   ->  true
2 5 9 14 20 , x = 10  ->  false
7 , x = 7             ->  true
7 , x = 3             ->  false
```

Prima apariție 8.2

Primești un vector ordonat crescător, cu valori care se repetă. Scrie o funcție care întoarce poziția primei apariții a lui x, sau -1 dacă nu apare.

`int primaPozitie(int v[], int n, int x)`

```
1 3 3 3 8 , x = 3  ->  1
1 3 3 3 8 , x = 8  ->  4
1 3 3 3 8 , x = 5  ->  -1
3 3 3 , x = 3      ->  0
7 , x = 7          ->  0
```

Unde ar trebui pusă 8.3

Primești un vector ordonat crescător și o valoare x. Scrie o funcție care întoarce poziția pe care ar trebui inserat x ca vectorul să rămână ordonat. Dacă x există deja, îl pui înaintea lui.

`int pozitiaDeInserare(int v[], int n, int x)`

```
2 5 9 14 , x = 7   ->  2
2 5 9 14 , x = 1   ->  0
2 5 9 14 , x = 20  ->  4
7 , x = 7          ->  0
```

Rădăcina pătrată întreagă 8.4

Scrie o funcție care întoarce partea întreagă a rădăcinii pătrate a lui n (n ≤ 10000), căutând binar răspunsul între 0 și n.

`int radical(int n)`

```
17     ->  4
16     ->  4
9999   ->  99
10000  ->  100
1      ->  1
0      ->  0
```

Câte sunt în interval 8.5

Primești un vector ordonat crescător și două valori a și b. Scrie o funcție care întoarce câte elemente sunt între a și b, inclusiv, fără să parcurgi tot vectorul.

`int cateInInterval(int v[], int n, int a, int b)`

```
2 5 9 14 20 , a = 5 , b = 14  ->  3
1 3 3 3 8 , a = 3 , b = 3     ->  3
2 5 9 , a = 10 , b = 20       ->  0
2 5 9 , a = 0 , b = 100       ->  3
7 , a = 7 , b = 7             ->  1
```

9. Palindromul pe vector · `palindrom-vector.h`

Studiezi din `algoritmi.h` : `estePalindromVector`

Oglindește vectorul 9.1

Scrie o funcție care inversează ordinea elementelor vectorului, schimbându-le două câte două din capete.

`void oglindeste(int v[], int n)`

```
1 2 3 4 5  ->  5 4 3 2 1
1 2 3 4    ->  4 3 2 1
4 4 4      ->  4 4 4
7          ->  7
```

Câte perechi nu se potrivesc 9.2

Scrie o funcție care întoarce câte perechi (v[i], v[n - 1 - i]) au elementele diferite. Elementul din mijloc nu face pereche.

`int perechiDiferite(int v[], int n)`

```
1 2 3 2 5  ->  1
1 2 3 4    ->  2
1 2 1      ->  0
7          ->  0
```

Aceeași sumă 9.3

Scrie o funcție care spune dacă toate perechile (v[i], v[n - 1 - i]) au aceeași sumă. Elementul din mijloc formează pereche cu el însuși.

`bool sumeEgale(int v[], int n)`

```
1 4 3 6    ->  true
1 2 3 4 5  ->  true
1 2 3 5    ->  false
3 3        ->  true
5          ->  true
```

Marginea comună 9.4

Scrie o funcție care întoarce câte perechi (v[i], v[n - 1 - i]) cu elemente egale găsești mergând dinspre capete spre mijloc, până la prima pereche diferită.

`int margineComuna(int v[], int n)`

```
1 2 5 2 1  ->  2
1 2 3 4 1  ->  1
3 2 3 5    ->  0
1 2 2 1    ->  2
4 4 4      ->  1
7          ->  0
```

Câte elemente adaug 9.5

Scrie o funcție care întoarce de câte elemente ai nevoie, adăugate la sfârșit, ca vectorul să devină palindrom. Elementele adăugate sunt copiile celor de la început, în ordine inversă.

`int elementeDeAdaugat(int v[], int n)`

```
1 2 3 2  ->  1
1 2 3    ->  2
2 2 1    ->  2
1 2 1    ->  0
5        ->  0
```

10. Divide et impera · `divide-et-impera.h`

Studiezi din `algoritmi.h`: `maximInjumatatire, mergeSort`

Amplitudinea 10.1

Scrie o funcție care întoarce diferența dintre maximul și minimul vectorului, înjumătățind intervalul. Se apelează cu st = 0 și dr = n - 1.

`int amplitudine(int v[], int st, int dr)`

```
7 2 9 4 5  ->  7
1 9        ->  8
4 4 4      ->  0
6          ->  0
```

De câte ori apare maximul 10.2

Scrie o funcție care întoarce de câte ori apare valoarea maximă în vector, înjumătățind intervalul.

`int aparitiiMaxim(int v[], int st, int dr)`

```
3 9 2 9 1  ->  2
4 4 4      ->  3
1 2 3      ->  1
5          ->  1
```

Ordonat prin înjumătățire 10.3

Scrie o funcție care spune dacă vectorul e ordonat crescător: ambele jumătăți ordonate, iar ultimul element din stânga nu depășește primul din dreapta.

`bool esteOrdonat(int v[], int st, int dr)`

```
1 3 7 9  ->  true
1 2 4 3  ->  false
1 7 3    ->  false
2 2 2    ->  true
5        ->  true
```

Sortare descrescătoare 10.4

Scrie o funcție care ordonează vectorul descrescător, prin interclasare. Vectorul are cel mult 100 de elemente.

`void sorteazaDescrescator(int v[], int st, int dr)`

```
3 1 4 1 5  ->  5 4 3 1 1
1 2 3      ->  3 2 1
2 2        ->  2 2
7          ->  7
```

Inversiunile 10.5

O inversiune e o pereche de poziții i < j cu v[i] > v[j]. Scrie o funcție care le numără în timpul interclasării; la final vectorul poate rămâne ordonat. Vectorul are cel mult 100 de elemente.

`int inversiuni(int v[], int st, int dr)`

```
2 4 1 3 5  ->  3
5 4 3 2 1  ->  10
2 2 1      ->  2
1 2 3      ->  0
7          ->  0
```

11. Turnurile din Hanoi · `hanoi.h`

Studiezi din `algoritmi.h` : `hanoi`

Câte mutări 11.1

Scrie o funcție care întoarce câte mutări are rezolvarea cu n discuri, fără să le afișezi.

`int nrMutari(int n)`

```
1   ->  1
3   ->  7
5   ->  31
10  ->  1023
0   ->  0
```

Câte mutări are discul d

11.2

Discurile sunt numerotate de la 1 (cel mai mic) la n (cel mai mare). Scrie o funcție care întoarce de câte ori se mută discul d, 1 ≤ d ≤ n.

`int mutariAleDiscului(int n, int d)`

```
n = 3 , d = 1  ->  4
n = 3 , d = 2  ->  2
n = 3 , d = 3  ->  1
n = 4 , d = 2  ->  4
n = 1 , d = 1  ->  1
```

Câte mutări de la x la y

11.3

Tijele sunt 1, 2, 3; discurile pornesc de pe tija 1 și ajung pe tija 3. Scrie o funcție care întoarce de câte ori un disc este mutat direct de pe tija x pe tija y.

`int cateMutari(int n, int x, int y)`

```
n = 3 , x = 1 , y = 3  ->  3
n = 3 , x = 1 , y = 2  ->  1
n = 3 , x = 3 , y = 1  ->  0
n = 4 , x = 1 , y = 3  ->  3
n = 1 , x = 1 , y = 3  ->  1
n = 0 , x = 1 , y = 3  ->  0
```

Mutarea numărul k

11.4

Scrie o funcție care afișează mutarea cu numărul k (de la 1) din rezolvarea cu n discuri, de pe tija 1 pe tija 3, sub forma tija sursă -> tija destinație, fără să afișezi celelalte mutări.

`void mutareaNumarul(int n, int k)`

```
n = 3 , k = 1  ->  1 -> 3
n = 3 , k = 2  ->  1 -> 2
n = 3 , k = 4  ->  1 -> 3
n = 3 , k = 7  ->  1 -> 3
n = 1 , k = 1  ->  1 -> 3
```

Ce disc se mută la pasul k

11.5

Scrie o funcție care întoarce numărul discului mutat la mutarea k (de la 1) din rezolvarea cu n discuri.

`int discMutat(int n, int k)`

```
n = 3 , k = 1  ->  1
n = 3 , k = 2  ->  2
n = 3 , k = 4  ->  3
n = 3 , k = 6  ->  2
n = 3 , k = 7  ->  1
n = 1 , k = 1  ->  1
```

12. Cifre cu void și referință · `void-referinta.h`

Studiezi din `algoritmi.h` : `sumaSiProdus, maximulCifrelor`

Pare și impare 12.1

Scrie o funcție care numără cifrele pare și cifrele impare ale unui număr n > 0. main pornește ambele contoare de la 0.

`void numaraPareImpare(int n, int& pare, int& impare)`

```
4738  ->  pare = 2 , impare = 2
1357  ->  pare = 0 , impare = 4
100   ->  pare = 2 , impare = 1
2     ->  pare = 1 , impare = 0
```

Minim și maxim 12.2

Scrie o funcție care află cea mai mică și cea mai mare cifră a unui număr n > 0. main pornește minimul de la 9 și maximul de la 0.

`void minimSiMaxim(int n, int& minim, int& maxim)`

```
4738  ->  minim = 3 , maxim = 8
1000  ->  minim = 0 , maxim = 1
5555  ->  minim = 5 , maxim = 5
7     ->  minim = 7 , maxim = 7
```

Înlocuiește o cifră 12.3

Scrie o funcție care înlocuiește în n toate aparițiile cifrei c1 cu cifra c2. Numărul n este parametru prin referință: după apel, n are valoarea nouă. n > 0.

`void inlocuiesteCifra(int& n, int c1, int c2)`

```
n = 72717 , c1 = 7 , c2 = 3  ->  n = 32313
n = 1200 , c1 = 0 , c2 = 5   ->  n = 1255
n = 1234 , c1 = 9 , c2 = 5   ->  n = 1234
n = 5050 , c1 = 5 , c2 = 0   ->  n = 0
n = 7 , c1 = 7 , c2 = 0      ->  n = 0
```

Separă pare de impare 12.4

Scrie o funcție care construiește numărul format din cifrele pare ale lui n și numărul format din cifrele impare, fiecare în ordinea inițială. Dacă nu există cifre de un fel, numărul respectiv e 0. main pornește ambele de la 0.

`void separaPareImpare(int n, int& pare, int& impare)`

```
4738   ->  pare = 48 , impare = 73
12345  ->  pare = 24 , impare = 135
1357   ->  pare = 0 , impare = 1357
1020   ->  pare = 20 , impare = 1
2468   ->  pare = 2468 , impare = 0
```

Fără cifra maximă

12.5

Scrie o funcție care șterge din n toate aparițiile cifrei maxime. n este parametru prin referință. n > 0.

`void taieMaxima(int& n)`

```
38738  ->  n = 373
4738   ->  n = 473
1020   ->  n = 100
1111   ->  n = 0
7      ->  n = 0
```

Pentru fiecare funcție pe care o scrii, verific-o pe:

1. cazul de oprire: **n = 0**, **n = 1** sau un vector cu **un singur** element;
2. un caz mic, pe care îl urmărești pe hârtie, **apel cu apel**;
3. cazul în care ce cauți **nu există**;
4. o valoare mai mare: dacă programul **nu se mai oprește** sau se închide singur, ți-a lipsit condiția de oprire sau nu te apropii de ea la fiecare apel.